HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

FINAL PROJECT REPORT

Topic: Parallelization Methods for CNN-Based Computer Vision Inference

Course: Parallel and Distributed Programming

Course code: IT4130E

Class/session: 168069 - Friday morning, periods 1–4, C7-215

Lecturer: Nguyen Tuan Dung

Group members: Luu Hieu An - 202400093
Nguyen Phu An - 20235466
Pham Chi Bang - 20235477
Nguyen Thanh Lam - 20235518

Hanoi, 2026

Contents

Abstract **1**

1 Introduction **2**

2 Problem Description **3**
2.1 CNN-Based Computer Vision Inference . . . 3
2.2 YOLO Video People Counting . . . 3
2.3 Parallel-Programming Focus . . . 3
2.4 Correctness Targets . . . 4

3 Parallelization Methods **5**
3.1 Method 1: Task Parallel YOLO Inference . . . 5
3.1.1 Temporal-Spatial Decomposition . . . 5
3.1.2 1D Mapping and Static Scheduling . . . 5
3.1.3 Communication and Merging . . . 5
3.1.4 YOLO Algorithm Sketch . . . 6
3.2 Weighted Process Placement . . . 6
3.3 Metric Collection . . . 6
3.4 Load Balancing and Metrics . . . 6

4 Experimental Setup **7**
4.1 Repository Evidence . . . 7
4.2 Physical Cluster . . . 7
4.3 Software and Data . . . 7
4.4 Metrics . . . 8
4.5 Experiment Matrix . . . 8
4.6 Recommended Additional Method 1 Experiments . . . 9

5 Results and Discussion **10**
5.1 YOLO Parallel Correctness . . . 10
5.2 Detector Accuracy Under Tile Splitting . . . 10
5.3 Input-Size Selection . . . 11
5.4 Granularity and Load Balance . . . 11
5.5 Static vs Dynamic Scheduling . . . 12
5.6 Speedup and Efficiency . . . 12

5.7 Weighted Static Placement 13
5.8 Local-Only and Full-Cluster Observation 13
5.9 Summary of Findings 14

6 Conclusion **15**

7 Member Contributions **16**

Abstract

This report drafts the final structure for the project *Parallelization Methods for CNN-Based Computer Vision Inference*. The final scope focuses on one implemented method: a YOLO/OpenMPI video people-counting pipeline that parallelizes inference at task level over video frames and image tiles.

The report uses measured values already recorded in the project reports. The YOLO pipeline runs on a three-MacBook physical cluster with C++17 and OpenMPI. A video is decomposed temporally into frames and spatially into tiles; each frame-tile pair becomes an MPI task. The final reported pipeline emphasizes static block-cyclic scheduling, rank-local YOLO workers, coordinate remapping, duplicate removal, global NMS, CSV output, and per-rank timing. Dynamic master-worker scheduling is kept only as a comparison point because this workload shows that it increases communication and performs worse.

The recorded YOLO experiments verify serial-versus-MPI correctness on 30 frames with zero mismatched frames. The detector accuracy experiment on 300 MOT17-derived frames reports mean absolute count error 7.983 and predicted average count 8.223 versus ground-truth average 16.207. The input-size experiment selects $N = 600$ frames because the 12-process runtime is 123.667 seconds, satisfying the required two-to-three-minute workload. On $2N = 1200$ frames, the 12-process weighted 4/6/2 run reaches 129.852 seconds and 2.662x wall-clock speedup. Static scheduling outperforms dynamic scheduling on the measured 600-frame 5x4 workload, and weighted static placement improves the idle-gap indicator from 0.466 to 0.235.

Chapter 1

Introduction

Computer vision inference is a useful setting for studying parallel programming because the workload is computationally heavy, data-rich, and naturally decomposable. A CNN-based detector can be applied repeatedly across frames and image regions. These repeated operations expose the main concerns of a parallel system: decomposition, mapping, communication, load balancing, correctness, and speedup.

This project focuses on CNN-based inference rather than model training. The main implemented application is video people counting with YOLO. Given an input video, the system predicts people bounding boxes and frame-level counts. The expensive work is detector inference over many frames and image tiles. The parallel contribution is the C++17/OpenMPI runtime around the detector: it creates tasks, schedules them across MPI ranks, launches local detector workers, gathers detections, removes duplicates, and writes experiment artifacts.

The report separates two types of correctness. Model accuracy means whether YOLO counts match human ground truth from MOT17. Parallel correctness means whether the MPI implementation produces the same result as the serial version of the same pipeline. A detector can undercount in crowded scenes while the parallel implementation remains correct. The measured results from the repository show this distinction clearly: serial and MPI counts match in the correctness test, while detector-versus-ground-truth accuracy remains limited on crowded MOT17 frames.

The main objectives are:

- define CNN-based computer vision inference as a parallel workload;
- present task parallelism for YOLO video inference;
- report measured YOLO correctness, input-size, scheduling, granularity, speedup, and weighted-placement results from the repository;
- propose additional Method 1 experiments that can make the final report deeper without adding a second algorithmic direction.

Chapter 2

Problem Description

2.1 CNN-Based Computer Vision Inference

The problem is to execute CNN-based video inference faster by distributing repeated detector work across MPI processes. The program applies a fixed pretrained model; it does not train or update model weights. The parallel system must preserve the output of the serial inference pipeline while reducing runtime or revealing the cost of communication.

2.2 YOLO Video People Counting

The input video is a finite sequence of frames:

$$V = \{F_1, F_2, \ldots, F_N\}. \tag{2.1}$$

The target output for each frame is the number of detected people after post-processing:

$$count(F_i) = |\{b \mid b \text{ is a final person bounding box in } F_i\}|. \tag{2.2}$$

The detector backend is a pretrained YOLO model through a local Python worker process [2]. The MPI program does not split YOLO neural-network kernels. Instead, MPI distributes repeated YOLO calls over frame-tile tasks.

For a tile grid with R rows and C columns, frame F_i is decomposed into:

$$F_i \to \{T_{i,1}, T_{i,2}, \ldots, T_{i,R\times C}\}. \tag{2.3}$$

A task is one frame-tile inference:

$$task(i,j) = T_{i,j}, \qquad |\mathcal{T}| = N \times R \times C. \tag{2.4}$$

Rank 0 converts tile-local detections back to full-frame coordinates, applies tile ownership filtering, removes duplicates, and writes final frame counts.

2.3 Parallel-Programming Focus

The final report studies one implemented MPI pipeline in depth instead of splitting attention across two methods. The YOLO pipeline exercises the following parallel-programming concepts:

Table 2.1: Parallel-programming concepts in the YOLO pipeline

Concept	YOLO MPI implementation
Parallelism level	Task parallelism across repeated YOLO inference calls
Decomposition	Hybrid temporal-spatial decomposition into frame-tile tasks
Mapping	1D flattened task list with static block-cyclic ownership
Communication	Rank-local inference followed by centralized detection and metric gathering
Load balancing	Tile granularity, static versus dynamic scheduling, and weighted process placement
Correctness target	MPI frame counts match serial frame counts after the same post-processing pipeline

2.4 Correctness Targets

The serial YOLO baseline processes the same frame-tile task list without MPI distribution and applies the same post-processing pipeline. Parallel correctness is:

$$\Delta_i = |count_{parallel}(F_i) - count_{serial}(F_i)|. \tag{2.5}$$

The correctness experiment reports mismatched frames, maximum count error, and mean count error. Detector accuracy against MOT17 ground truth is evaluated separately because model error and parallelization error are different questions.

Chapter 3

Parallelization Methods

3.1 Method 1: Task Parallel YOLO Inference

The YOLO pipeline uses task-level parallelism over independent frame-tile inference tasks. The implementation is centered on `src/yolo_mpi_cpp.cpp` in the repository. Each MPI rank runs the same C++ program, and rank-local detector work is delegated to a long-lived Python YOLO worker.

3.1.1 Temporal-Spatial Decomposition

The input video is first decomposed by time into frames and then by space into image tiles. If the video has N frames and each frame is split into an $R \times C$ grid, the total number of tasks is NRC. Larger grids create more tasks and can improve scheduling flexibility, but they increase duplicate detections near tile borders and increase post-processing work.

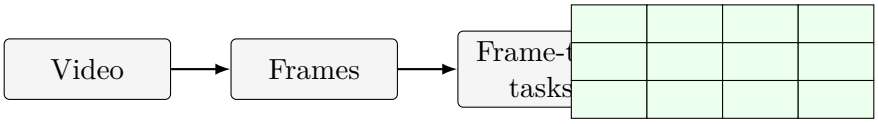

Figure 3.1: Hybrid temporal-spatial decomposition for YOLO video inference.

3.1.2 1D Mapping and Static Scheduling

All frame-tile tasks are flattened into a one-dimensional list. For task index i, chunk size k, and P ranks, static block-cyclic scheduling assigns:

$$owner(task_i) = \left\lfloor \frac{i}{k} \right\rfloor \bmod P. \tag{3.1}$$

Static scheduling has low dispatch overhead because ranks can determine their own tasks locally. The measured repository results show that this advantage matters for the 600-frame 5x4 tiled YOLO workload.

3.1.3 Communication and Merging

After local inference, rank 0 gathers detections and rank metrics from all ranks. It then remaps tile-local boxes into frame coordinates, applies tile-owner filtering, runs global non-maximum suppression, and computes per-frame counts. This centralized merge is necessary because people near tile boundaries may be detected by multiple tiles.

3.1.4 YOLO Algorithm Sketch

Algorithm 1 Static MPI YOLO people-counting pipeline

1: Build task list $\mathcal{T}$ from N frames and an $R \times C$ tile grid.
2: **for** each MPI rank r in parallel **do**
3: Select tasks whose static owner is r.
4: Run local YOLO inference for selected tasks.
5: Store detections and timing metrics.
6: **end for**
7: Gather serialized detections and rank metrics to rank 0.
8: Rank 0 remaps boxes, removes duplicates, computes frame counts, and writes CSV outputs.

3.2 Weighted Process Placement

The physical cluster is heterogeneous: the three machines do not have identical CPU throughput. Equal rank placement is simple, but it can leave faster machines waiting for slower ones. Weighted process placement keeps the same static task-mapping algorithm while changing the number of MPI ranks launched on each host. Stronger machines receive more ranks and therefore more positions in the block-cyclic task stream; the slower machine receives fewer ranks.

The report evaluates uniform 4/4/4 placement and weighted placements such as 3/6/3 and 4/6/2. This isolates process placement from the scheduling algorithm: if runtime and idle gap improve, the reason is the host-to-rank assignment rather than a different task dispatch policy.

3.3 Metric Collection

Each run records both output correctness and timing behavior. The most important timing quantities are wall-clock runtime, runtime without communication, per-rank computation time, communication time, and the idle-gap indicator. These values are needed because speedup alone can hide the reason for poor performance. For this workload, a run can have many tasks and still be slow if rank 0 spends too much time collecting detections or if a slower node receives too much work.

3.4 Load Balancing and Metrics

The report uses:

$$T_r = T_{compute,r} + T_{comm,r} + T_{idle,r}, \tag{3.2}$$

and the idle-gap indicator reported by the project scripts. The course criterion treats the run as balanced when the idle gap is at most 0.25. The measured YOLO results show that equal process placement does not meet this criterion, while the best weighted static placement does.

Chapter 4

Experimental Setup

4.1 Repository Evidence

The draft is based on the local repository `yolo-mpi-people-count`. The strongest source for measured YOLO values is `reports/final_report_hust_style.md`, with additional full-cluster discussion in `reports/additional_experiments_20260623.md`. The artifact branch also provides compact CSV files under `reports/artifacts/`; the Method 1 CSV artifacts have been copied into this report repository under `artifacts/`.

4.2 Physical Cluster

The project uses three physical MacBook machines connected through a local network. The benchmark mode uses CPU execution to match the course emphasis on MPI processes and processor assignment. GPU/MPS execution is kept for demonstration and live-camera use rather than formal CPU speedup measurement.

Table 4.1: Physical cluster roles

Host	Machine type	Role in experiments
master	MacBook Air M4	Coordinates execution, gathers results, stores outputs, and can run local tasks.
node1	MacBook Pro M4	Worker host; measured results suggest it can sustain more assigned ranks.
node2	MacBook Air M2	Worker host; weighted placement assigns fewer ranks to reduce imbalance.

4.3 Software and Data

The implementation uses C++17, OpenMPI, Python helper scripts, and a pretrained Ultralytics YOLO detector. The main dataset source is MOT17-derived video data [1]. Compact MOT17 subsets are used for quick correctness and granularity experiments; longer sequence segments are used for the required input-size and speedup runs.

Table 4.2: Main YOLO experimental configuration

Item	Setting
Detector	Pretrained YOLO, person-class filtering
Parallel framework	C++17 and OpenMPI
Benchmark device	CPU
Dataset	MOT17-derived sequences
Main process count	12 MPI processes
Main long-run grid	5x4 regions
Schedulers	Static block-cyclic for the final pipeline; dynamic master-worker only as a comparison baseline
Placement variants	Uniform 4/4/4 and weighted 3/6/3, 4/6/2

4.4 Metrics

The report uses the following metrics from the project reports and generated CSV design:

- serial-versus-MPI correctness: mismatched frames and count error;
- detector accuracy: MAE, RMSE, percentage error, exact-match ratio;
- runtime with communication: wall-clock time including coordination;
- runtime without communication: compute-oriented time reported by the scripts;
- speedup and efficiency:

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p}; \tag{4.1}$$

- idle-gap indicator for load-balance analysis.

4.5 Experiment Matrix

Table 4.3: Experiment matrix for final report completion

Experiment	Method	Status in this draft	Report purpose
Parallel correctness	YOLO	Filled from report values	Verify MPI output against serial baseline.
Detector accuracy	YOLO	Filled from report values	Separate model accuracy from parallel correctness.
Input-size selection	YOLO	Filled from report values	Choose N near 120–180 s.
Granularity	YOLO	Filled from report values	Study task count, communication, and idle time.
Scheduling	YOLO	Filled from report values	Compare static and dynamic scheduling.
Speedup	YOLO	Filled from report values	Measure S_p and E_p.
Weighted placement	YOLO	Filled from report values	Address heterogeneous physical cluster.

4.6 Recommended Additional Method 1 Experiments

Since the final scope focuses on the YOLO MPI method, the report can be strengthened by adding more experiments around the same pipeline rather than introducing a second algorithm. Table 4.4 lists useful follow-up measurements.

Table 4.4: Recommended follow-up experiments for the YOLO MPI method

Experiment	Suggested variables	Purpose
Tile-grid sweep on $N = 600$	1x1, 2x2, 4x3, 5x4, 6x4	Find the best accuracy/runtime trade-off under the official workload size.
Weighted placement sweep	5/5/2, 6/4/2, 4/6/2, 3/7/2	Show how heterogeneous rank assignment changes idle gap and runtime.
Chunk-size sensitivity	chunk size 1, 2, 4, 8, 16	Check whether block-cyclic granularity changes load balance without changing tile grid.
Communication breakdown	gather time, serialization time, merge/NMS time	Explain where overhead comes from when task count increases.
Repeated-run stability	at least 3 repeats for the main rows	Report mean and standard deviation for defense-quality reliability.
Accuracy under tiling	baseline, 2x2, 4x3, 5x4, 6x4	Quantify whether finer tiles harm detector output through border effects.

Chapter 5

Results and Discussion

5.1 YOLO Parallel Correctness

Table 5.1 reports the serial-versus-MPI correctness result recorded in the repository report. The MPI run produced the same frame counts as the serial baseline on the tested sequence.

Table 5.1: YOLO serial-versus-MPI correctness

Result	Frames	Mismatched frames	Max count error	Mean error	Evidence
Passed	30	0	0	0.000	repository report

This validates the parallel pipeline for the tested configuration. It indicates that task splitting, MPI result collection, coordinate remapping, duplicate removal, and final counting preserve the serial pipeline output.

5.2 Detector Accuracy Under Tile Splitting

Table 5.2 compares detector accuracy against MOT17 ground-truth counts for the normal single-machine YOLO baseline and for different MPI tile-splitting configurations. The purpose of this experiment is to check whether spatial tiling changes the final detection accuracy, not only whether the MPI implementation is faster.

Table 5.2: YOLO accuracy comparison across baseline and tile-splitting configurations

Configuration	Grid	Frames	MAE	RMSE	Percent error	Exact match	Pred. avg.
Single-machine YOLO baseline	1x1	300	7.983	8.269	0.490	0.000	8.223
MPI tiled YOLO	2x2	300	7.983	8.269	0.490	0.000	8.223
MPI tiled YOLO	4x3	300	8.223	8.517	0.504	0.000	7.977
MPI tiled YOLO	5x4	300	8.383	8.682	0.514	0.000	7.812

The 2x2 configuration is effectively equivalent to the single-machine baseline, so moderate spatial

tiling does not change detector behavior. Finer grids introduce a small degradation: the 4x3 configuration is about 3 percent worse than baseline, and the 5x4 configuration is about 5 percent worse. This suggests that aggressive tile splitting can harm accuracy through border effects and imperfect cross-tile merging, even when it creates more parallel work.

5.3 Input-Size Selection

The compact MOT17 subset is useful for quick runs but does not reach the required two-to-three-minute runtime. Table 5.3 also reports the total number of frame-tile tasks and the wall-clock time per task, making it easier to compare efficiency across input sizes. The long-sequence experiment selects $N = 600$ frames because the 12-process wall-clock runtime is 123.667 seconds.

Table 5.3: YOLO input-size selection

Frames	P	Grid	Tasks	Runtime with comm. (s)	Runtime without comm. (s)	ms/task	Selection note
30	12	4x3	360	9.165	2.999	25.46	Compact quick test
60	12	4x3	720	12.570	6.801	17.46	Compact quick test
100	12	4x3	1200	17.776	11.596	14.81	Compact quick test
150	12	4x3	1800	23.585	17.103	13.10	Compact quick test
300	12	5x4	6000	54.764	49.156	9.13	Long sequence
600	12	5x4	12000	123.667	114.352	8.97	Selected N
837	12	5x4	16740	146.316	139.136	8.74	Best ms/task, longer validation
1200	12	5x4	24000	129.852	128.122	5.41	$2N$, weighted run

The time-per-task column shows that very small inputs have high overhead because startup, synchronization, and aggregation are amortized over too few tasks. As the long-sequence input grows from 300 to 1200 frames, the measured time per task generally improves, indicating better amortization of fixed overhead. The 1200-frame row uses the weighted 4/6/2 run because it is the $2N$ speedup workload. The 600-frame run is selected as N because it is the first long-sequence case that satisfies the required 120–180 second runtime window.

5.4 Granularity and Load Balance

Table 5.4 summarizes the official granularity experiment using weighted 6/4/2 placement. The weighted placement keeps the idle-gap indicator at 0.235, within the 25 percent course criterion. Increasing the number of regions still increases task count and communication, so granularity remains a performance trade-off even when load balance is acceptable.

Table 5.4: YOLO granularity and load balance with weighted 6/4/2 placement

Grid	P	Tasks	Max comp. (s)	Avg comp. (s)	Total comm. (s)	Idle gap	Balance
1x1	12	150	0.731	0.483	10.674	0.235	Balanced
2x2	12	600	3.155	2.205	16.306	0.235	Balanced
4x3	12	1800	9.266	6.375	34.881	0.235	Balanced
5x4	12	3000	13.733	9.133	51.790	0.235	Balanced

The result shows that weighted placement can satisfy the idle-gap criterion, but simply making tasks finer is not enough to guarantee faster execution. Finer tiles create more scheduling opportunities, but they also increase intermediate detections, communication, and master-side merging. This motivates the scheduler and weighted-placement experiments.

5.5 Static vs Dynamic Scheduling

The full-cluster scheduler comparison used 12 processes, 600 frames, a 5x4 grid, and CPU execution. Static scheduling is much faster on this measured workload, so the dynamic master-worker method is not used as the main report algorithm.

Table 5.5: Static versus dynamic scheduling on 600 frames

Scheduler	P	Grid	Runtime with comm. (s)	Runtime without comm. (s)	Imbalance	Idle gap	Balance
Static	12	5x4	40.619	36.522	1.202	0.466	Not balanced
Dynamic	12	5x4	102.316	96.959	2.778	0.837	Not balanced

Dynamic scheduling is inefficient for this specific task. The 5x4 grid creates many small frame-tile tasks, so a master-worker queue forces rank 0 to repeatedly send task metadata, poll or receive worker results, and dispatch the next task. That extra communication and coordination is larger than the load-balancing benefit. Static block-cyclic scheduling avoids per-task dispatch traffic, lets ranks select their own work locally, and is therefore used for the final weighted-placement experiment.

5.6 Speedup and Efficiency

The speedup experiment uses the selected input size doubled to $2N = 1200$ frames and applies weighted static placement for the multi-rank runs. This keeps the process distribution consistent with the heterogeneous cluster: stronger machines receive more ranks, while the slower machine receives fewer ranks.

Table 5.6: YOLO speedup on 1200 frames with weighted placement

P	Runtime with comm. (s)	Runtime without comm. (s)	Wall speedup	Efficiency	Note
1	345.677	342.467	1.000	1.000	Baseline
2	201.111	205.100	1.719	0.859	Weighted placement
4	178.238	181.289	1.939	0.485	Weighted placement
8	146.212	146.204	2.364	0.296	Weighted placement
12	129.852	128.122	2.662	0.222	Weighted 4/6/2

The speedup is measurable but not linear. Under weighted placement, the wall-clock speedup

rises from 1.719x at two processes to 2.662x at twelve processes. The 12-process computation-only speedup is approximately 2.673x. The improvement comes from reducing idle gap and assigning fewer ranks to the slower node.

The follow-up report also records two complete 1200-frame repeats. Those values are useful for defense because they show stable 12-process runtime but a variable one-process baseline:

Table 5.7: Two-repeat speedup summary on 1200 frames with weighted placement

P	Repeats	Mean runtime with comm. (s)	Runtime stdev (s)	Mean speedup	Note
1	2	317.314	40.111	1.000	Baseline varied strongly
2	2	216.984	22.447	1.462	Weighted placement
4	2	197.756	27.604	1.605	Weighted placement
8	2	157.824	16.421	2.011	Weighted placement
12	2	129.756	0.135	2.445	Weighted 4/6/2

5.7 Weighted Static Placement

The equal 4/4/4 placement does not meet the 25 percent idle-gap criterion. The best measured weighted placement assigns four ranks to the master, six to node1, and two to node2.

Table 5.8: Weighted static placement on heterogeneous cluster

Placement	Distribution	Runtime with comm. (s)	Runtime without comm. (s)	Idle gap	Balance
Uniform	4/4/4	40.619	36.522	0.466	Not balanced
Weighted	3/6/3	34.712	30.339	0.371	Not balanced
Weighted	4/6/2	29.581	27.484	0.235	Balanced

This result supports the claim that processor assignment matters on a heterogeneous physical cluster. Equal rank placement gives every host the same number of processes, but the machines do not have equal throughput. Weighted placement reduces runtime and satisfies the load-balance criterion.

5.8 Local-Only and Full-Cluster Observation

Additional experiments show that increasing process count on a single machine can saturate local resources. On the master-only 600-frame 5x4 run, 2 processes achieved 288.494 seconds while 4 and 8 processes were slower. In contrast, the full three-machine 600-frame dynamic sweep improved from 202.715 seconds at 1 process to 90.900 seconds at 12 processes. This supports the defense point that physical distribution matters more than only increasing the number of local MPI ranks.

5.9 Summary of Findings

The measured YOLO results support the main task-parallel narrative. The implementation preserves serial output, reaches the required input-size runtime, shows measurable speedup, and demonstrates that scheduling and rank placement strongly affect performance. The strongest measured improvement in load balance comes from weighted static placement, which reduces the idle-gap indicator from 0.466 to 0.235. The remaining report work should deepen Method 1 with additional placement, granularity, and communication-breakdown experiments rather than adding a second method.

Chapter 6

Conclusion

This report drafts the final content for *Parallelization Methods for CNN-Based Computer Vision Inference* using evidence from the `yolo-mpi-people-count` repository. The completed YOLO method demonstrates task-level parallelism with hybrid temporal-spatial decomposition, 1D task mapping, static block-cyclic scheduling, rank-local detector execution, and centralized detection merging.

The measured YOLO results are strong enough for the main experimental narrative. The serial-versus-MPI correctness test passes with zero mismatched frames. The selected 600-frame workload runs in 123.667 seconds with 12 processes, satisfying the required two-to-three-minute range. The official 1200-frame weighted 4/6/2 run reaches 129.852 seconds and 2.662x wall-clock speedup. Static scheduling outperforms dynamic scheduling on the measured 600-frame 5x4 workload because dynamic dispatch increases communication for many small tasks, while weighted static placement improves the idle-gap indicator to 0.235 and satisfies the 25 percent load-balance criterion.

The detector accuracy result should be interpreted separately from parallel correctness. YOLO undercounts crowded MOT17 frames, with MAE 7.983 on the recorded 300-frame accuracy experiment. This is a model limitation, not evidence that MPI changed the pipeline output.

Overall, the project covers the core course concepts through one coherent method: decomposition, mapping, communication topology, scheduling, load balancing, correctness verification, input-size selection, and speedup measurement. The best next step is to make Method 1 deeper by adding more placement, granularity, communication-breakdown, and repeated-run evidence.

Chapter 7

Member Contributions

Table 7.1 records a draft responsibility split based on the current repository contents. The group should update it with confirmed final work before submission.

Table 7.1: Draft member contributions

Member	Student ID	Contribution
Luu Hieu An	202400093	Report integration, repository evidence audit, final presentation alignment, and result table preparation.
Nguyen Phu An	20235466	Cluster setup, OpenMPI execution, hardware evidence collection, and hostfile/placement experiments.
Pham Chi Bang	20235477	C++/MPI implementation, YOLO scheduling pipeline, and result aggregation support.
Nguyen Thanh Lam	20235518	Plot generation, speedup table preparation, experimental discussion, and final report polishing.

Bibliography

[1] MOTChallenge. Mot17 benchmark. https://motchallenge.net/data/MOT17/, 2026.

[2] Ultralytics. Ultralytics yolo documentation. https://docs.ultralytics.com/, 2026.